

TRƯỜNG THPT PHAN ĐÌNH PHÙNG

BẢN THUYẾT MINH
MÔ HÌNH SẢN PHẨM THAM DỰ CUỘC THI
SÁNG TẠO THANH THIẾU NIÊN NHI ĐỒNG
TOÀN QUỐC

1. Tên mô hình, sản phẩm dự thi:

“Hybrid Mamba-2+Attention World Model cho robot manipulation”

2. Lĩnh vực dự thi:

- Phần mềm tin học

3. Thông tin về tác giả, giáo viên hướng dẫn:

Tác giả:

- Họ và tên: Dương Thanh Hoan

- Lớp: 11A5.

- Trường: THPT Phan Đình Phùng –

- Số điện thoại: 0396923143 - Email: thoan4965@gmail.com

Giáo viên hướng dẫn:

- Họ và tên: Võ Thị Thanh Trúc - Số điện thoại: 0836013539

Tháng 8/ 2026

BẢN THUYẾT MINH

MÔ HÌNH SẢN PHẨM THAM DỰ CUỘC THI SÁNG TẠO THANH, THIẾU NIÊN, NHI ĐỒNG TOÀN QUỐC LẦN THỨ 22 (NĂM 2026)

I. Thông tin chung

Trường	Nội dung
Tên mô hình, sản phẩm dự thi	"Hybrid Mamba-2+Attention World Model cho robot manipulation"
Lĩnh vực dự thi	Phần mềm tin học
Tác giả	Dương Thanh Hoan — Lớp 11A5 — THPT Phan Đình Phùng — SĐT: 0396923143 — Email: thoan4965@gmail.com
Giáo viên hướng dẫn	Võ Thị Thanh Trúc — SĐT: 0836013539

Mục lục

MỤC LỤC

1. Mở đầu	2
2. Liên quan	2
3. Phương pháp	3
3.1. Kiến trúc tổng thể	3
3.2. Hình A — So sánh 3 kiến trúc bộ dự đoán	3
3.3. Hàm mất mát	7
3.4. Vì sao chọn Mamba-2	7
4. Hành trình thực nghiệm	7
4.1. V0 — Robot thật: so sánh CfC và AR	7
4.2. V1 — Hybrid CfC+Attention: phát hiện quan trọng	9
4.3. V2.1 — Hybrid Mamba-2+Attention	10
5. Kết quả	10
5.1. TwoRoom	10
5.2. Push-T	11
5.3. Đồ thị huấn luyện	12
5.4. Thử nghiệm với các tầm nhìn dài hơn	12
5.5. Thời gian giải kế hoạch bằng CEM	12
6. Thảo luận	13
6.1. ODE vs Discrete State — nguồn gốc khác biệt	13
6.2. Sai số tích lũy ở tầm nhìn lớn — giới hạn chung	14
6.3. So sánh với LeWM	14
6.4. Hạn chế	14
6.5. Cân nhắc khi triển khai	14
7. Kết luận và hướng nghiên cứu tiếp nối	15
7.1. Kết luận	15
7.2. Hướng nghiên cứu tiếp nối: robot nhặt rác di động	15
8. Cách sử dụng, vận hành	17
8.1. [V0 — Robot thật] Vận hành phần cứng theo cử chỉ (demo)	17
8.1.2. [V0 — Robot thật] Vận hành mô hình thể giới + CEM trên robot thật	17
8.2. [V2.1 — Benchmark mô phỏng] Tái lập kết quả đánh giá	18
9. Tài liệu tham khảo	19

1. Mở đầu

Một câu để nhớ: Robot này biết nhìn, biết nghĩ — và đang học cách tự lập kế hoạch trước khi hành động.

Lập kế hoạch hành động từ ảnh camera là bài toán cốt lõi của robot: quan sát môi trường, dự đoán hậu quả của hành động, rồi chọn chuỗi hành động tối ưu. Joint Embedding Predictive Architecture (JEPA, LeCun [1]) mở ra hướng thay thế trực tiếp: học dự đoán trong **không gian tiềm ẩn**—gọn, giàu thông tin—thay vì tái tạo từng pixel.

LeWorldModel (Maes et al. 2026) [2] hiện thực hóa JEPA với kết quả cạnh tranh (TwoRoom 87%, Push-T 96%, Cube 74%, Reacher 88%) ở 15 triệu tham số. Tuy nhiên bộ dự đoán AR (Autoregressive Transformer, bộ dự đoán tự hồi quy) của LeWM là **không trạng thái**—mỗi bước chỉ dựa trên cửa sổ 3 khung hình—nên sai số tích lũy khi tầm nhìn kế hoạch tăng, như chính bài báo thừa nhận: *"chuỗi dự đoán tự hồi quy tích lũy sai số dự đoán khi tầm nhìn kéo dài"*. Chính vì vậy, thực nghiệm tại H=5 cho thấy khoảng cách đáng kể: LeWM AR đạt 88% trong khi kiến trúc lai có trạng thái trong nghiên cứu này đạt **94.7% ± 3.1%** (kết quả chi tiết tại §5).

Hành trình đến kết quả đó không thẳng một đường: tôi bắt đầu bằng giả thuyết bộ dự đoán **có trạng thái** sẽ chạy chuỗi dài tốt hơn, kiểm chứng trên **robot thật tay bionic 8 khớp tự chế**; sau đó chạm phải một hiện tượng bất ngờ khi đưa CfC (trạng thái ODE liên tục) vào JEPA—không phải do CfC yếu, mà do **tương tác giữa SIGReg và trạng thái ODE**—và cuối cùng thay thành phần đó bằng **Mamba-2** (Dao & Gu 2024) [4], với kiến trúc $6 \times \{\text{Self-Attention(AdaLN)} \rightarrow \text{Mamba-2}\}$. Toàn bộ hành trình này được trình bày tại §4.

Ba đóng góp chính:

1. **Lấy mẫu theo lịch trình (Scheduled Sampling) cho mạng ODE liên tục** — kỹ thuật lần đầu được áp dụng cho loại mô hình này (chi tiết §4.1).
2. **Phát hiện tương tác SIGReg × trạng thái ODE** — nhiều chống suy biến bị khuếch đại bởi động học liên tục, nguyên nhân thất bại của kiến trúc lai đầu tiên (chi tiết §4.2).
3. **Kiến trúc lai block-level Mamba-2+Attention** — lần đầu tiên Attention và SSM kết hợp trong cùng một khối (Jamba/TransMamba chỉ lai ở mức tầng), lần đầu Mamba-2 trạng thái rời rạc làm bộ dự đoán trong mô hình thế giới JEPA (kết quả §5).

2. Liên quan

JEPA (LeCun 2022) [1]. Joint Embedding Predictive Architecture học cách dự đoán trong không gian tiềm ẩn thay vì tái tạo pixel. Gồm bộ mã hóa (ảnh → vector tiềm ẩn) và bộ dự đoán (vector tiềm ẩn hiện tại + hành động → vector tiềm ẩn tương lai). Hàm mất mát gồm lỗi dự đoán + một cơ chế chống suy biến (SIGReg, VICReg, hoặc stop-grad + EMA).

LeWorldModel (Maes et al. 2026) [2]. Hiện thực hóa JEPA cho mô hình thế giới điều khiển robot. Bộ mã hóa ViT-tiny 5M, bộ dự đoán Transformer $6 \times \{\text{Self-Attn} \rightarrow \text{MLP}\}$ 10M. Chống suy biến bằng SIGReg [5] — phép thử Epps-Pulley trên các phép chiếu ngẫu nhiên, ép vector tiềm ẩn về phân phối Gaussian. Hai hàm mất mát: lỗi dự đoán + $\lambda \cdot \text{SIGReg}$. Đạt 87% TwoRoom, 96% Push-T, 74% Cube, 88% Reacher. Hạn chế: bộ dự đoán không trạng thái — sai số tích lũy khi tầm nhìn tăng.

CfC (Hasani et al. 2022) [3]. Mạng nơ-ron độ sâu liên tục dạng đóng (Closed-form Continuous-depth network) — xấp xỉ nghiệm đúng của phương trình vi phân LTC. Không cần bộ giải số, có trạng thái ẩn liên tục theo thời gian. Vượt trội thời gian so với RNN/AR (V0: độ trôi 34 lần thấp hơn AR). Bài báo CfC [3] thừa nhận: "*CfC có thể gặp vấn đề gradient tiêu biến cho phụ thuộc dài hạn*" — cần hỗn hợp bộ nhớ hoặc Lấy mẫu theo lịch trình.

Mamba-2 (Dao & Gu 2024) [4]. Mô hình trạng thái chọn lọc (selective state space model) — cải tiến từ Mamba-1 [8] với thuật toán SSD, tận dụng tensor cores, nhanh hơn 2-8 lần. Trạng thái rời rạc, không khuếch đại nhiễu như ODE. Tuy nhiên, Ma & Najarian [6] chứng minh toán học: phụ thuộc xa của Mamba giảm theo **hàm mũ** (suy giảm bộ nhớ hàm mũ) — thông tin càng xa càng mờ.

Hybrid Attention + SSM. Jamba [9]: xen kẽ Attention và Mamba theo từng tầng. TransMamba [10]: chuyển đổi Attention/SSM theo độ dài chuỗi. Cả hai đều lai ghép ở mức **tầng**, chưa có kiến trúc lai ghép **trong cùng một khối** (Attention + SSM trong cùng block như công trình này).

Giới hạn chung của mô hình thế giới tiềm ẩn. LeWM bài báo [2] thừa nhận: "*lập kế hoạch với các mô hình thế giới tiềm ẩn hiện nay vẫn bị giới hạn trong tầm nhìn ngắn*". Sai số tích lũy khi tăng tầm nhìn lập kế hoạch là hạn chế chung của lớp mô hình này.

3. Phương pháp

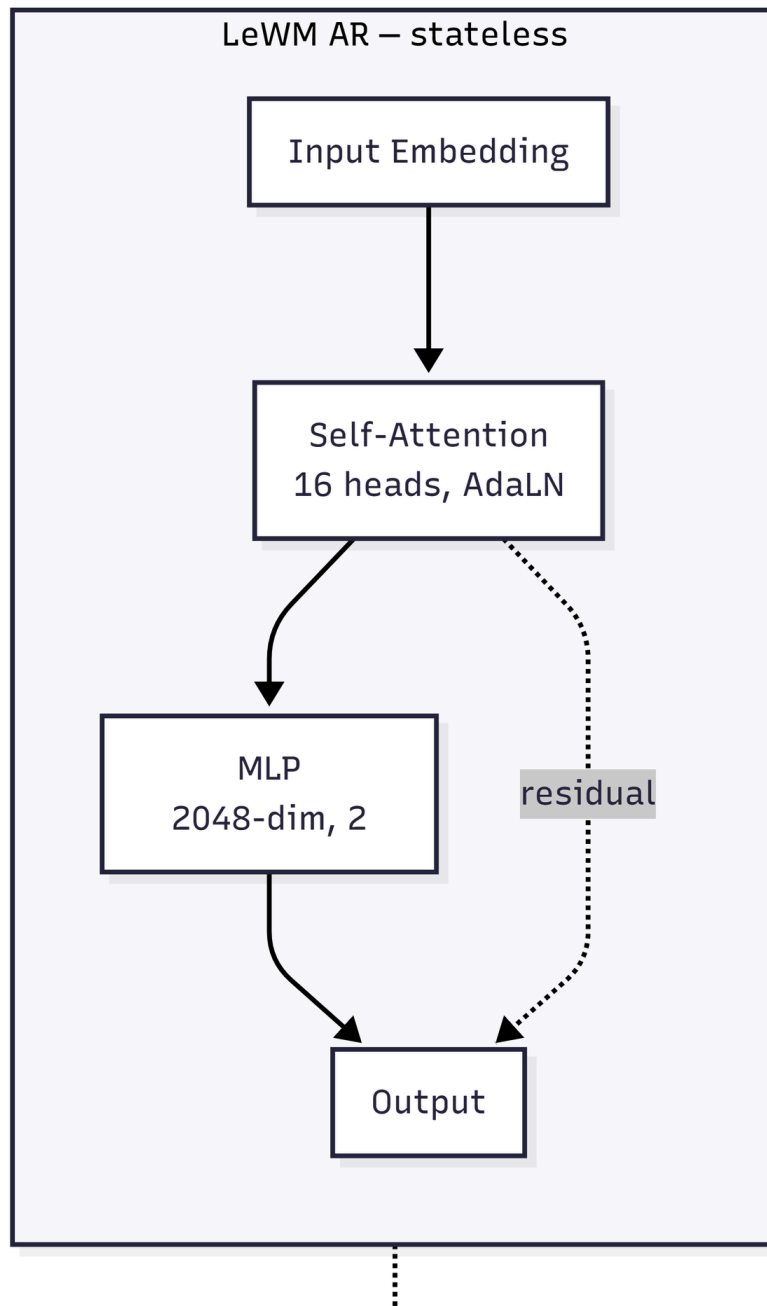
3.1. Kiến trúc tổng thể

Mô hình JEPA gồm 3 thành phần: **Bộ mã hóa** ViT-tiny (5M), patch 14, ảnh 224×224 → CLS token 192 chiều; **Bộ dự đoán** 6 block transformer với mặt nạ nhân quả, điều khiển theo hành động qua AdaLN; **Bộ chiếu**: MLP 2 lớp (192→2048→192) + BatchNorm1d.

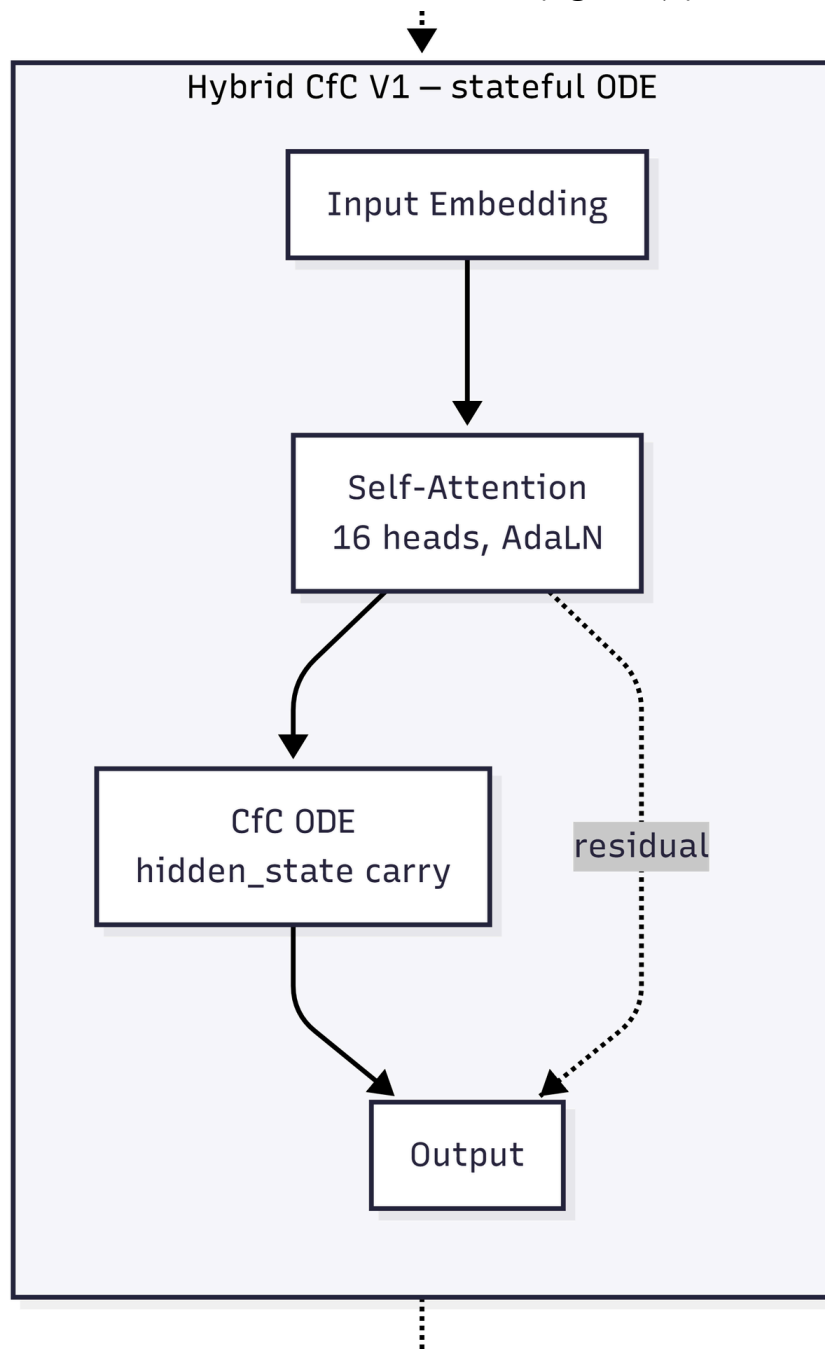
3.2. Hình A — So sánh 3 kiến trúc bộ dự đoán

Ba kiến trúc dự đoán dùng chung khung xử lý (input embedding → self-attention → thành phần lỗi → output), chỉ khác **thành phần lỗi**:

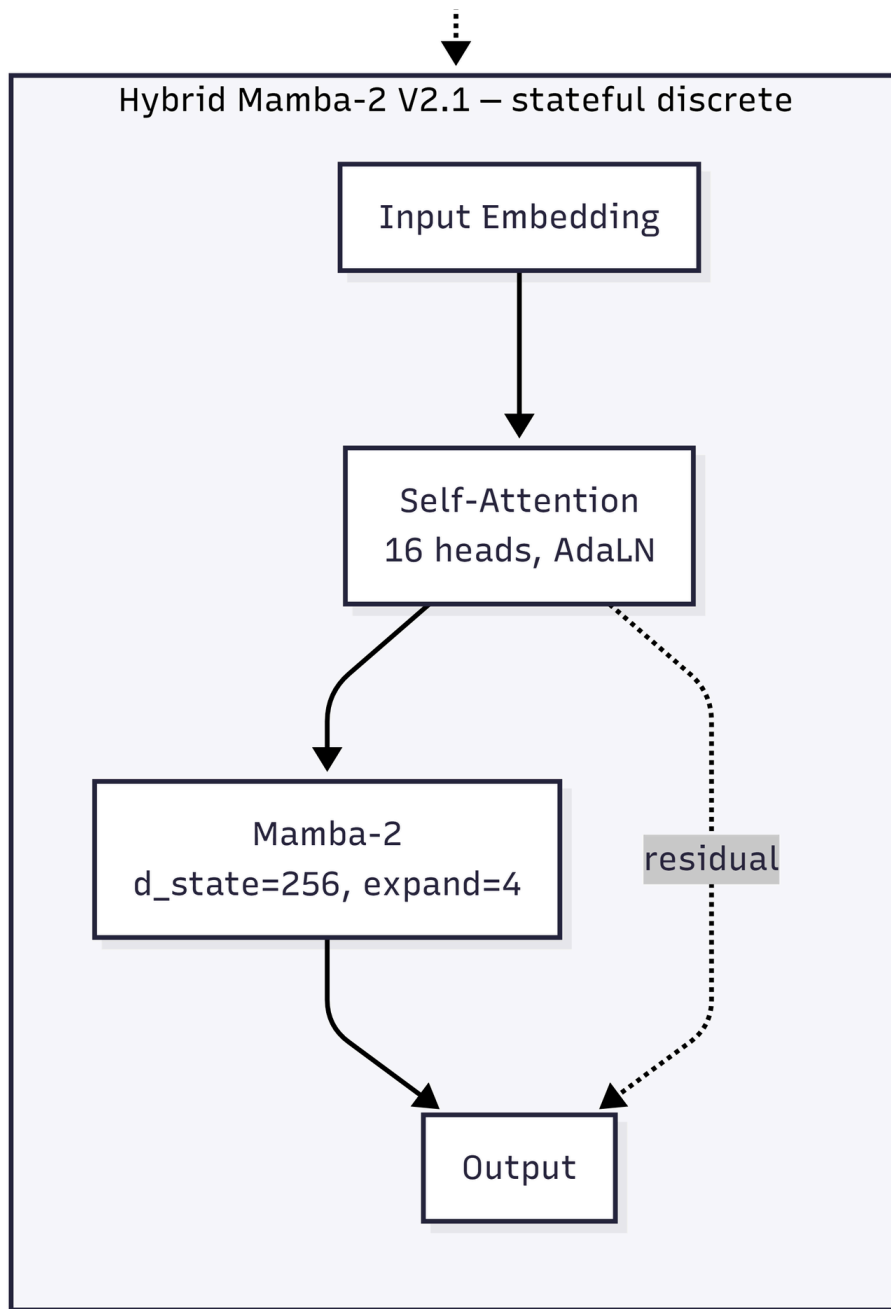
Hình A1: LeWM AR — bộ dự đoán Attention + MLP không trạng thái



Hình A2: Hybrid CfC V1 — Attention + CfC ODE có trạng thái (bị nhiễu $\times$ ODE)

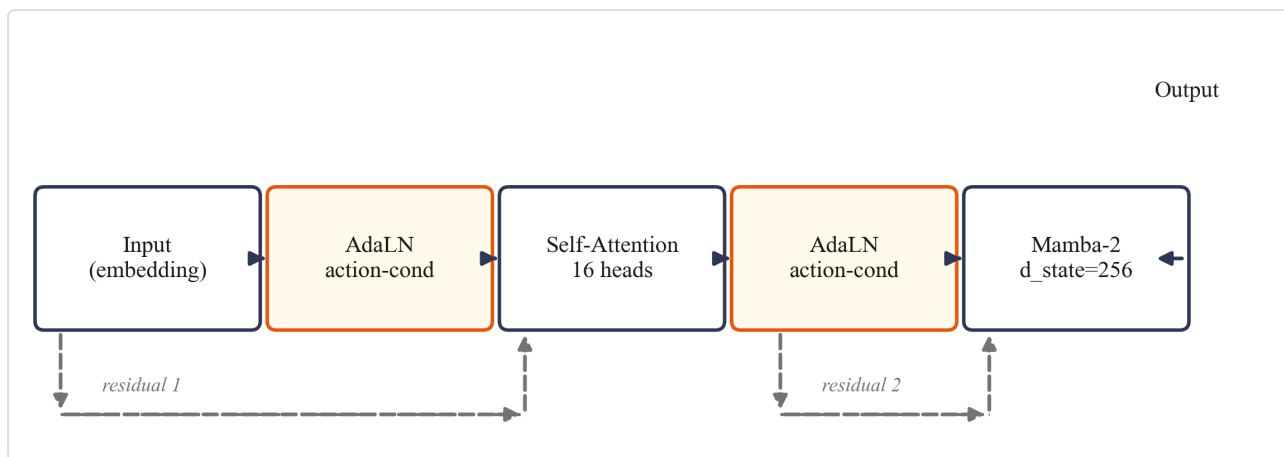


Hình A3: Hybrid Mamba-2 (đề xuất) — Attention + Mamba-2 có trạng thái rời rạc



Mỗi block gồm: (1) AdaLN-modulation — mã hóa hành động + scale/shift từ action embedding; (2) Self-Attention — đa đầu (16 heads, $d_{im_head}=64$), causal mask; (3) Feed-forward — thay đổi tùy phiên bản.

Kiến trúc block (Mamba2ConditionalBlock):



Hình A4: Sơ đồ khối Mamba2ConditionalBlock — AdaLN điều hòa theo hành động, 2 nhánh residual (màu xám đứt nét)

AdaLN nhận action embedding, xuất 6 giá trị (shift×3 + scale×3 + gate×3). Khởi tạo zero để hành động có hiệu lực dần.

Tham số	Giá trị	Ghi chú
depth	6	Số block, bằng LeWM
heads	16	Attention heads
dim_head	64	
d_state	256	SSM state size (LeWM dùng 128)
expand	4	Expansion factor (Mamba-2 internal)

3.3. Hàm mất mát

Hàm mất mát LeWM: $L = \text{MSE}(\text{pred_emb} - \text{tgt_emb}) + \lambda \cdot \text{SIGReg}(\text{embeddings})$, với MSE: ép dạy — dự đoán vector tiềm ẩn bước tiếp theo; SIGReg [5]: phép thử Epps-Pulley trên các phép chiếu ngẫu nhiên, ép vector tiềm ẩn về phân phối Gaussian; $\lambda = 0.09$ từ bài báo LeWM [2].

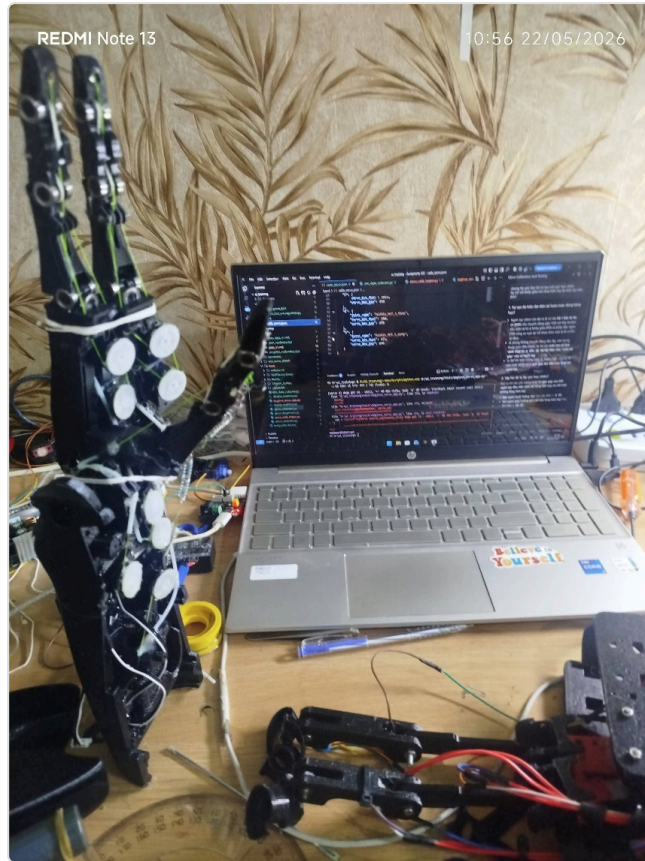
3.4. Vì sao chọn Mamba-2

1. **Trạng thái rời rạc** — không khuếch đại nhiễu như ODE CfC
2. **Khởi tạo HiPPO [4]** — giá trị riêng bị ràng buộc → gradient ổn định (bằng chứng lý thuyết)
3. **Quét chọn lọc** — cổng gating phụ thuộc đầu vào → lọc nhiễu
4. **Hệ sinh thái hoàn chỉnh** — gói PyPI, kernel Triton tối ưu, tích hợp HuggingFace — cho phép thí nghiệm nhanh giả thuyết mà không tốn thời gian build từ nguồn. Đây là yếu tố quan trọng vì V1 (CfC) từng mất nhiều thời gian sửa lỗi kernel ODE, làm chậm vòng nghiên cứu.

4. Hành trình thực nghiệm

4.1. V0 — Robot thật: so sánh CfC và AR

Tôi xây dựng **tay bionic 8 khớp** từ servo SC09, khung in 3D, camera webcam — thiết kế tham khảo DexHand (Rob Knight, 2022) [12]. Mục tiêu: so sánh AR và CfC trên cùng pipeline robot thật.



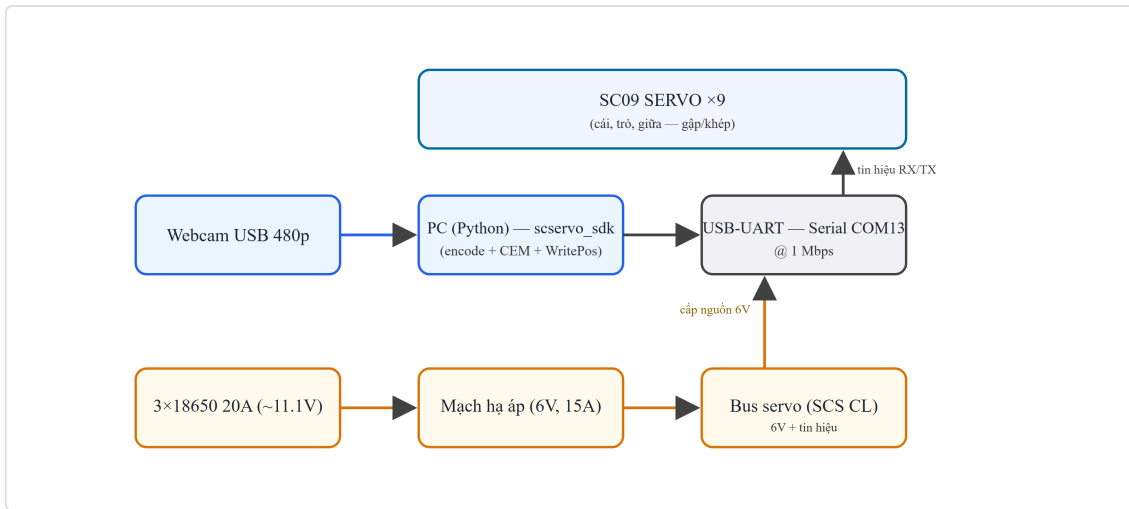
Hình B1: Robot tay bionic 8 khớp — vị trí nghỉ, khung in 3D, 8 servo SC09

Vật liệu chế tạo

Bảng 1: Vật liệu và nguồn gốc

Linh kiện	Model	Chức năng	Nguồn gốc
Servo	SC09 (Waveshare)	8 khớp $\times$ 3 ngón đối kháng, bus SCS CL, 300°, 0-1023	Mua mới, linh kiện phổ biến
Khung	PLA in 3D	Kết nối khớp, tham khảo DexHand V1	In tại nhà
Camera	Webcam USB 480p	Quan sát môi trường, CAP_DSHOW	Mua mới, linh kiện phổ biến
Chuyển đổi	USB-UART adapter	UART $\leftrightarrow$ USB + cấp nguồn servo	Mua mới
Nguồn	3 $\times$ 18650	Qua mạch hạ áp $\rightarrow$ 6V	Mua mới

Toàn bộ linh kiện mua được tại các cửa hàng điện tử thông thường — mô hình có thể tái lập với chi phí và dụng cụ hợp lý.



Hình B2: Sơ đồ kết nối phần cứng

Dữ liệu tự thu thập: box kín camera cố định, đèn LED phơi sáng cố định, nền trắng; ~50 episode từ tư thế nghỉ đến lúc nắm, ~8900 frame; tăng cường màu ColorJitter $\rightarrow$ ~17800 frame. Phát hiện nắm thành công bằng sai số vị trí |lệnh–thực tế|<100.

Kết quả V0:

Mô hình	Batch	SS	Độ trôi cuộn dự đoán	Chênh lệch dự đoán–thực tế
AR-32	32	—	0.2485	71×
AR-264	264	—	0.0012	1.2×
CfC V4	32	—	0.0025	2×
CfC V3	264	0 $\rightarrow$ 30%	0.0795	66×

Kết luận V0: CfC vượt trội về mặt thời gian (độ trôi 0.000014/bước, 34× thấp hơn AR). Lấy mẫu theo lịch trình (Scheduled Sampling) cải thiện CfC 29× (0.072 $\rightarrow$ 0.0025) — chưa có công bố nào áp dụng kỹ thuật này cho mạng ODE liên tục trước đây. AR không dùng lịch trình, nhưng cần batch >128 để ổn định.

Lưu ý phân định: Kết quả V0 là kiểm chứng *nguyên lý* trên robot thật — dữ liệu tự thu nhỏ (8900 frame, 1 camera, 1 mô hình robot) — chứng minh cơ chế hoạt động (CfC độ trôi thấp hơn AR, chuỗi CEM nắm chạy được). **Đây không phải kết quả thống kê tái lập được.** Kết quả tái lập được (3 hạt giống $\times$ 2 GPU) là V2.1: 94.7% $\pm$ 3.1% (Push-T) — xem §5.2 và §8.2.

4.2. V1 — Hybrid CfC+Attention: phát hiện quan trọng

Thay bộ dự đoán AR bằng CfC trong JEPa: bộ dự đoán $6 \times \{\text{Tự chú ý} \rightarrow \text{CfC}\}$, điều hòa theo hành động qua AdaLN, hàm mất mát lỗi dự đoán + $\lambda \cdot \text{SIGReg}$ ($\lambda=0.09$, num_proj=1024).

Kết quả TwoRoom:

Budget	T	Success rate
50	16	78%
150	16	6%

Kết quả 78% (mục tiêu=25, ngân sách 50) → 6% (mục tiêu=100, ngân sách 150). CfC vẫn chạy tốt ở mục tiêu gần, nhưng chết ở mục tiêu xa. So sánh với V0 (cùng CfC + SIGReg $\lambda=0.05$, 20 bước cuộn dự đoán, ổn định): vấn đề là **nhiều SIGReg tích lũy qua trạng thái ẩn ODE**. Bài báo CfC [3] thừa nhận: "*CfC có thể gặp vấn đề gradient tiêu biến cho phụ thuộc dài hạn*". **Kết luận: CfC không tương thích với SIGReg. Cần trạng thái rời rạc.**

4.3. V2.1 — Hybrid Mamba-2+Attention

Thay CfC bằng Mamba-2 (trạng thái rời rạc, không khuếch đại nhiễu). Cùng kiến trúc $6 \times \{\text{Self-Attn} \rightarrow \text{Mamba-2}\}$, cùng hàm mất mát lỗi dự đoán + $\lambda \cdot \text{SIGReg}$, cùng cấu hình (trừ bộ dự đoán). Huấn luyện 10 epoch trên Vast RTX 5090 (32GB VRAM), batch 128, bf16, ~5h.

Kết quả và so sánh trên 2 GPU (T4, RTX 5090), 3 hạt giống — trình bày tại §5 (TwoRoom §5.1, Push-T §5.2); kiến trúc lai block-level cũng là đóng góp chính đã tóm tắt tại §1.

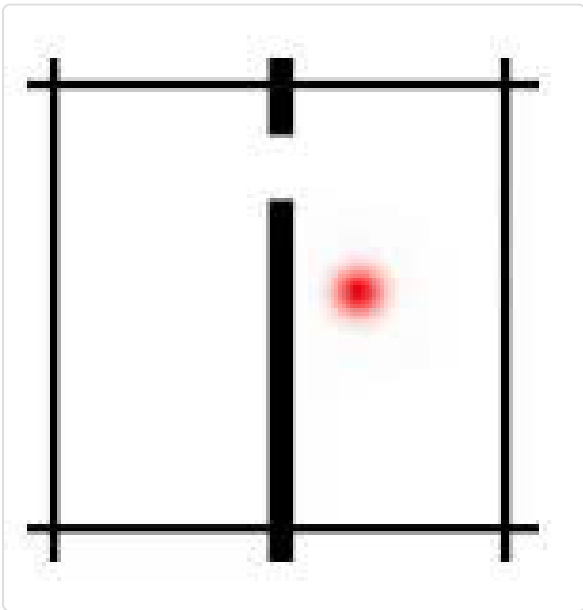
5. Kết quả

Các thử nghiệm và số liệu dưới đây là bằng chứng thật, mọi so sánh được thực hiện trong cùng điều kiện: cùng 3 hạt giống (seed), cùng cấu hình $H=K=5$, cùng script đánh giá; LeWM đối chiếu từ 3 nguồn (GitHub, bản chạy tại chỗ, bài báo).

5.1. TwoRoom

Mô hình	Ngân sách	Mục tiêu	Tỷ lệ thành công (3 seed)	Ghi chú
Hybrid Mamba-2	50	25	85.3% ± 10.1% (84,76,96)	T4 fp32, 3 seed
Hybrid Mamba-2	50	25	86.0% ± 10.0%	RTX 5090, 3 seed
LeWM chính thức	50	25	80.7% ± 10.3% (78,72,92)	T4 fp32, 3 seed
LeWM chính thức	50	25	81.3% ± 9.5%	RTX 5090, 3 seed
LeWM bài báo [2]	50	25	87%	Tham khảo (GPU khác)

So sánh ở mức ngân sách 150, mục tiêu=100: Hybrid đạt 18% (9/50).



Hình C: Môi trường TwoRoom. Nguồn: [2].

5.2. Push-T

Hạt giống	Tỷ lệ thành công	
3072	92% (46/50)	
3073	98% (49/50)	
3074	94% (47/50)	
Trung bình $\pm$ độ lệch chuẩn	94.7% $\pm$ 3.1%	

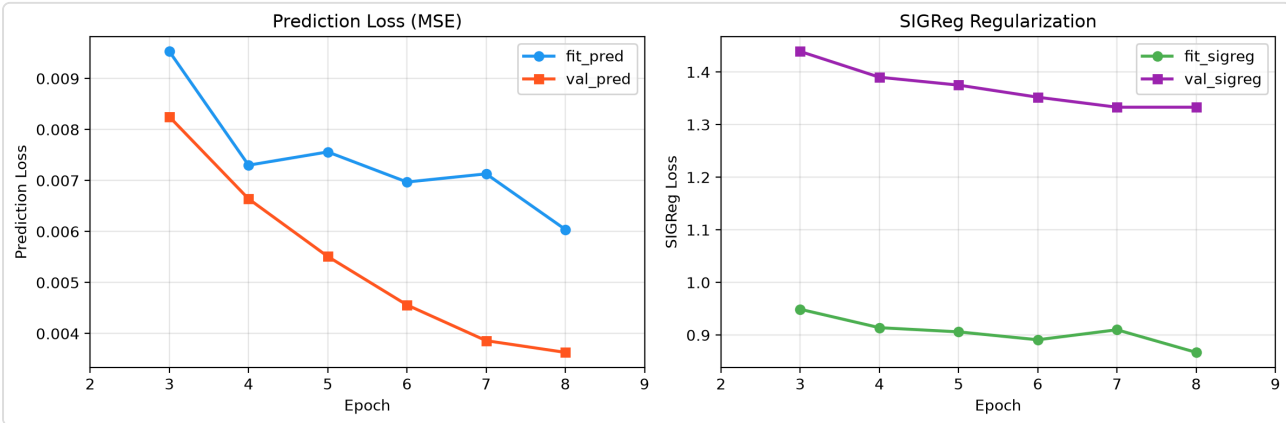
Mô hình	GPU	Tỷ lệ thành công	Hạt giống
Hybrid Mamba-2	T4 fp32	94.7% $\pm$ 3.1% (92,98,94)	3072,3073,3074
Hybrid Mamba-2	RTX 5090	94.7% $\pm$ 3.1% (94,98,92)	3072,3073,3074
LeWM chính thức (3 seed)	T4 fp32	86.0% $\pm$ 4.0%	3072,3073,3074
LeWM chính thức (3 seed)	RTX 5090	88.0% $\pm$ 4.0% (88,92,84)	3072,3073,3074
LeWM bài báo [2]	(không rõ)	96% $\pm$ 2.83%	—

Hybrid Mamba-2 vượt LeWM trên cùng GPU: +6.7% (5090), +8.7% (T4) với 3 seed cho cả hai mô hình. So với số bài báo (96% $\pm$ 2.83%), kết quả nằm trong khoảng tin cậy chồng lấp (có nghĩa: 94.7% không khác biệt thống kê so với 96% vì dải $\pm 3.1\%$ và $\pm 2.83\%$ thuộc nhau — kết quả tương đương, không thua kém). Kết quả **tái lập trên 2 GPU khác nhau (T4 + RTX 5090) đều 94.7%** — không phụ thuộc phần cứng.



Hình D: Môi trường Push-T. Nguồn: [2].

5.3. Đồ thị huấn luyện



Hình E: Training curve Push-T — *val_pred* giảm 57% qua 8 epoch; SIGReg plateau từ epoch 6

5.4. Thử nghiệm với các tầm nhìn dài hơn

Tầm nhìn	LeWM AR	Hybrid Mamba-2	Ghi chú
5	86.0% ± 4.0%	94.7% ± 3.1%	Hybrid vượt trội
10	40%	42%	Sai số tích lũy bắt đầu ảnh hưởng cả hai
20	4%	2%	Cả hai gần như ngẫu nhiên — giới hạn chung của mô hình thể giới tiềm ẩn

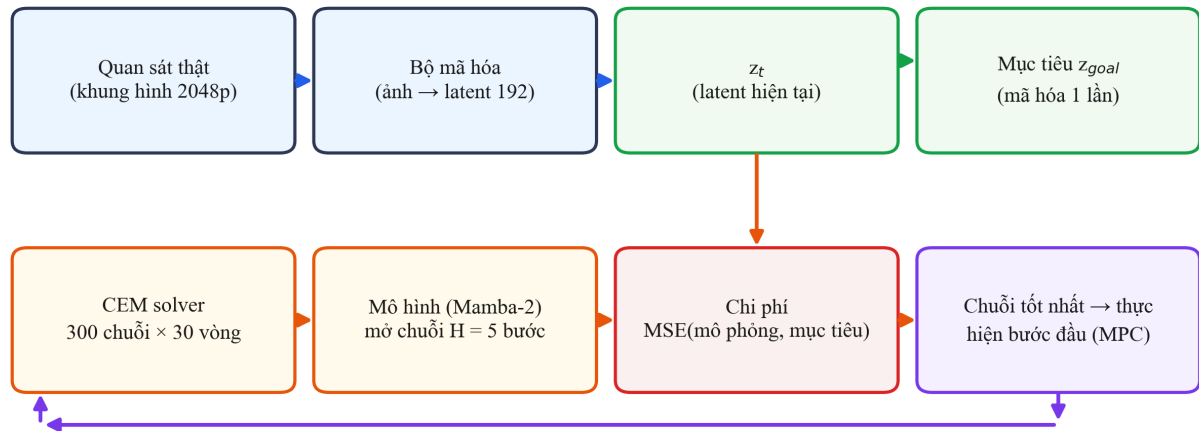
Bài báo LeWM [2] thừa nhận: "*chuỗi dự đoán tự hồi quy tích lũy sai số khi tầm nhìn kéo dài*" — và Hybrid Mamba-2 cũng chịu chung giới hạn này.

5.5. Thời gian giải kế hoạch bằng CEM

Mô hình	GPU	Lần đầu	Sau khi biên dịch / mỗi episode
LeWM AR (3 seed)	T4 fp32	~98s (khởi tạo cuDNN)	~20s
LeWM AR (3 seed)	RTX 5090	—	~43s/50ep
Hybrid Mamba-2	T4 fp32	~1160s (biên dịch kernel Triton lần đầu)	~85s
Hybrid Mamba-2	RTX 5090	—	~107s

Hybrid chậm hơn LeWM $\sim 4\times$ (T4) / $\sim 2.5\times$ (RTX 5090) — phân tích hạn chế xem §6.4.

CEM lập: 300 chuỗi hành động được mô phỏng trong MÔ HÌNH — không chạm robot thật



Hình F: CEM lập kế hoạch trong không gian tiềm ẩn — 300 chuỗi hành động được mô phỏng trong MÔ HÌNH

6. Thảo luận

6.1. ODE vs Discrete State — nguồn gốc khác biệt

Cấu hình	Mục tiêu	Ngân sách	CfC (T=16, heads=8)	Mamba-2 (T=4, heads=16)	Ghi chú
H=5, K=5	25	50	78%	86%	Cùng mục tiêu, ngân sách — CfC thấp hơn
H=5, K=5	100	150	6%	18%	Mục tiêu xa 4×, cả hai đều giảm

Khi mục tiêu xa hơn, cả CfC và Mamba đều giảm — CfC 78%→6%, Mamba 86%→18%. Dù dùng T và số đầu chú ý khác nhau (mỗi kiến trúc tối ưu riêng), ở cùng điều kiện đánh giá (mục tiêu=100, ngân sách=150) Mamba-2 (18%) cao hơn CfC (6%) — khoảng cách này phù hợp với phân tích ODE khuếch đại nhiễu SIGReg.

Cơ chế: SIGReg [5] tạo nhiễu trong các vector tiềm ẩn. Với AR (không trạng thái) và Mamba-2 (trạng thái rời rạc), nhiễu được làm mới hoặc lọc sau mỗi lần lập lại kế hoạch. Với CfC, nhiễu tích lũy trong trạng thái ẩn ODE qua nhiều lần lập kế hoạch. Lý thuyết ủng hộ nhận định: bài báo Neural SDE [11] ghi nhận "trạng thái đầu vào sai lệch nhẹ sẽ được khuếch đại qua các tầng sâu", trong khi bài báo Mamba [8] khẳng định "lọc chọn lọc các nhiễu không liên quan" — hai cơ chế đối lập giải thích kết quả quan sát.

Lưu ý: CfC không yếu về mặt thời gian — đã kiểm chứng trên robot thật (V0, xem §4.1). CfC chỉ không tương thích với SIGReg, không phải CfC kém.

6.2. Sai số tích lũy ở tầm nhìn lớn — giới hạn chung

Khi tăng tầm nhìn từ 5 lên 10, cả LeWM AR ($86.0\% \pm 4.0\% \rightarrow 40\%$) và Hybrid Mamba-2 ($94.7\% \pm 3.1\% \rightarrow 42\%$) đều giảm mạnh. Ở tầm nhìn 20, cả hai gần ngẫu nhiên (4% và 2%). Điều này xác nhận giới hạn của mô hình thể giới tiềm ẩn với lập kế hoạch dài hạn, như bài báo LeWM [2] thừa nhận trong phần hạn chế: *"lập kế hoạch với các mô hình thể giới tiềm ẩn hiện nay vẫn bị giới hạn trong tầm nhìn ngắn"*.

Nguyên nhân: cuộn dự đoán bước theo bước trong không gian tiềm ẩn không có hiệu chỉnh bằng quan sát — sai số từ dự đoán tích lũy qua bước $\rightarrow$ giá trị tính từ vector tiềm ẩn cuối sai $\rightarrow$ bộ lập kế hoạch chọn hành động sai. MPC (điều khiển dự đoán theo cửa sổ trượt) chỉ giảm thiểu, không loại bỏ. Đáng chú ý: cơ chế này giống Lấy mẫu theo lịch trình đã dùng cho CfC ở V0 — cả hai giải quyết sai số tích lũy bằng cách "thường xuyên hiệu chỉnh bằng số liệu đúng". Lịch trình ở huấn luyện, cửa sổ trượt ở lập kế hoạch — cùng một ý tưởng nhất quán.

Riêng với Mamba-2, Ma & Najarian [6] chứng minh phụ thuộc dài giảm theo hàm mũ — rào cản phụ cho planning dài hơn.

6.3. So sánh với LeWM

Kết quả của Hybrid nằm trong khoảng tin cậy chồng lấp với số bài báo ($96\% \pm 2.83\%$) — cho thấy kiến trúc KHÔNG thua kém dù bị hạn chế về phần cứng so với bài báo, và tái lập ổn định trên 2 GPU với cùng cấu hình (số liệu bảng §5.2, §5.4).

6.4. Hạn chế

1. **Thời gian giải kế hoạch CEM:** $\sim 85\text{s/lần}$ vs LeWM $\sim 20\text{s/lần}$ trên T4 (chậm $\sim 4\times$) và $\sim 107\text{s}$ vs $\sim 43\text{s}/50$ lần trên RTX 5090 (chậm $\sim 2.5\times$) — đã đo trên 2 GPU, cùng hạt giống và cấu hình. Đây là đánh đổi thực tế của kernel Mamba-2 so với kernel attention đã tối ưu sẵn của LeWM. Dự địa cải thiện: (i) dùng torch.compile để biên dịch tổng mô hình; (ii) chỉnh kernel Mamba-2 cho GPU cụ thể; (iii) hướng triển khai §7.2 bỏ hẳn bộ giải, thay bằng phản xạ 1 lần qua (vài mili giây).
2. **Suy giảm bộ nhớ Mamba:** Suy giảm hàm mũ của trạng thái Mamba [6] — cần số hạng tương tác để khắc phục.
3. **Suy giảm ở tầm nhìn 10,20:** Cả hai mô hình đều chịu sai số tích lũy — hạn chế chung của mô hình thể giới tiềm ẩn, chưa có giải pháp.
4. **Phương sai TwoRoom cao:** Độ lệch chuẩn $\pm 10.1\%$ (Hybrid) và $\pm 10.3\%$ (LeWM) với 3 hạt giống. Cần thêm hạt giống để khẳng định xu hướng.

6.5. Cân nhắc khi triển khai

Hạn chế về thời gian giải kế hoạch cần được cân nhắc khi triển khai:

- **Mamba-2 vốn dùng thuật toán SSD tận dụng tensor-core matmul [4]** — nhanh $2\text{-}8\times$ so với Mamba-1, và độ phức tạp xử lý **gần tuyến tính** với độ dài chuỗi, trong khi Attention có độ phức tạp $O(N^2)$. Lợi thế này thể hiện rõ khi độ dài chuỗi tăng (lập kế hoạch tầm nhìn lớn, nhiều khung hình).
- Tuy nhiên, **kết quả thời gian của nghiên cứu này đạt được trên T4 fp32 và RTX 5090** — kernel Mamba-2 khai thác tốt hơn trên GPU hiện đại (chậm $\sim 2.5\times$ trên 5090 so với $\sim 4\times$ trên T4). **Chi phí huấn luyện Mamba-2 so với AR trong nghiên cứu này chưa được đo** — cần so sánh có kiểm soát trên cùng phần cứng trước khi kết luận.

- Khi triển khai trên phần cứng hạn chế (robot phổ thông, CPU/MCU), hướng nghiên cứu tiếp nối **không dùng bộ giải CEM** — thay bằng phản xạ CfC-habit 1 lần qua (vài mili giây), chỉ dùng mô hình 1 bước để kiểm chứng (xem §7.2, §6.2). Các nghiên cứu gần đây (MambaLite-Micro, Quamba) ủng hộ khả năng triển khai mô hình trạng thái trên thiết bị biên với lượng từ hóa INT8 — là hướng đáng xem xét cho giai đoạn sau.

7. Kết luận và hướng nghiên cứu tiếp nối

7.1. Kết luận

Nghiên cứu đề xuất **Hybrid Mamba-2+Attention** — kiến trúc lai thay lớp trung gian không trạng thái (LeWM AR) bằng Mamba-2 trạng thái rời rạc trong bộ dự đoán JEPA, và kể một hành trình nghiên cứu trọn vẹn: từ robot thật (V0), qua một thất bại có giá trị (V1 — phát hiện SIGReg tương tác xấu với trạng thái ODE), đến kiến trúc thắng ở tầm nhìn 5 trong cùng điều kiện (V2.1: Push-T 94.7%±3.1% so với LeWM 88.0%±4.0% trên RTX 5090 và 86.0%±4.0% trên T4 — khoảng cách tái lập trên 2 GPU, 3 seed).

Trọn bộ ba đóng góp — Lấy mẫu theo lịch trình cho CfC (29×), phát hiện SIGReg × ODE, kiến trúc lai block-level — được trình bày ở §1 và chi tiết ở §4-§6. Trong viễn cảnh triển khai, thời gian giải kế hoạch chậm ~2.5× (RTX 5090) là đánh đổi có thể chấp nhận được để đổi lấy độ chính xác cao hơn, và được khắc phục một phần bằng chiến lược thay bộ giải bằng phản xạ nhanh (CfC-habit) ở hướng tiếp nối §7.2.

7.2. Hướng nghiên cứu tiếp nối: robot nhặt rác di động

Nghiên cứu này mở ra một ứng dụng cụ thể: **robot nhặt rác di động** cho trường học, khuôn viên công cộng và sân chơi — nơi rác nhỏ như vỏ chai, nylon còn rơi vãi hằng ngày. Đáng làm vì hai lý do: mô hình không đòi hỏi mua sản phẩm đắt đỏ mà được chế tạo từ linh kiện phổ thông mua tại cửa hàng điện tử, và chính cách "phản xạ + tự biết lạ + học an toàn" của nghiên cứu trả lời được bài toán robot ngoài môi trường thật: kiểm soát mạnh mẽ thiếu linh hoạt, hoặc linh hoạt nhưng mất đầu — thay vào đó là chọn mức tin cậy theo tình huống. Từ điểm nối tiếp này, robot có thể nhân rộng tại nhiều trường thay vì chỉ tồn tại một lần trong phòng thí nghiệm.

Bộ não ở Hình H chỉ là một nửa — nửa còn lại là phần thân dân dụng: camera S3-CAM 320×320, cảm biến quét laser LiDAR X1 cùng IMU MPU6050 nối về combo ESP32-S3 (MCU) — SLAM Toolbox dựng bản đồ, EKF phối hợp vị trí 50Hz, Nav2 điều hướng tránh vật cản; 3 khớp tay SC09 có cảm biến lực, 2 mô tơ bánh xe qua L298N, 2 cụm 3S + 2 mạch hạ áp 5V cho mỗi khối. Hai nửa nối với nhau qua chuỗi vận hành: mỗi lúc chỉ một bộ phận giữ lái, không tranh nhau.

Bản thiết kế dùng hai kết quả chính của nghiên cứu — cả hai là mạng nơ-ron CfC [3] (Closed-form Continuous-depth, mạng ODE trạng thái liên tục):

- **Phản xạ (CfC-habit)** — mạng nơ-ron nhỏ (20-75K tham số), học từ chính dữ liệu người điều khiển robot thật ở V0 (500 tập thao tác, chỉ lấy các lần thành công). Nó so sánh "lệnh người thao tác – lệnh điều khiển cơ bản" và ghi nhớ phần bù đó — như người lái xe giữ tay lái cơ bản thẳng, còn phần cảm tính chỉnh cho khéo chính là phản xạ học được. Nhờ đó, sau khi quen, robot chạy nhanh mà không cần tính toán lại từ đầu;
- **Dự đoán bước sau (CfC-imagination, mô hình 1 bước)** — mạng nơ-ron dự đoán trạng thái kế tiếp (lệnh sắp gửi → trạng thái của tay) từ mô hình thế giới của nghiên cứu. Nó chạy như tiểu não ở con người: liên tục dự đoán diễn biến tiếp theo của hành động; phải có dự đoán mới

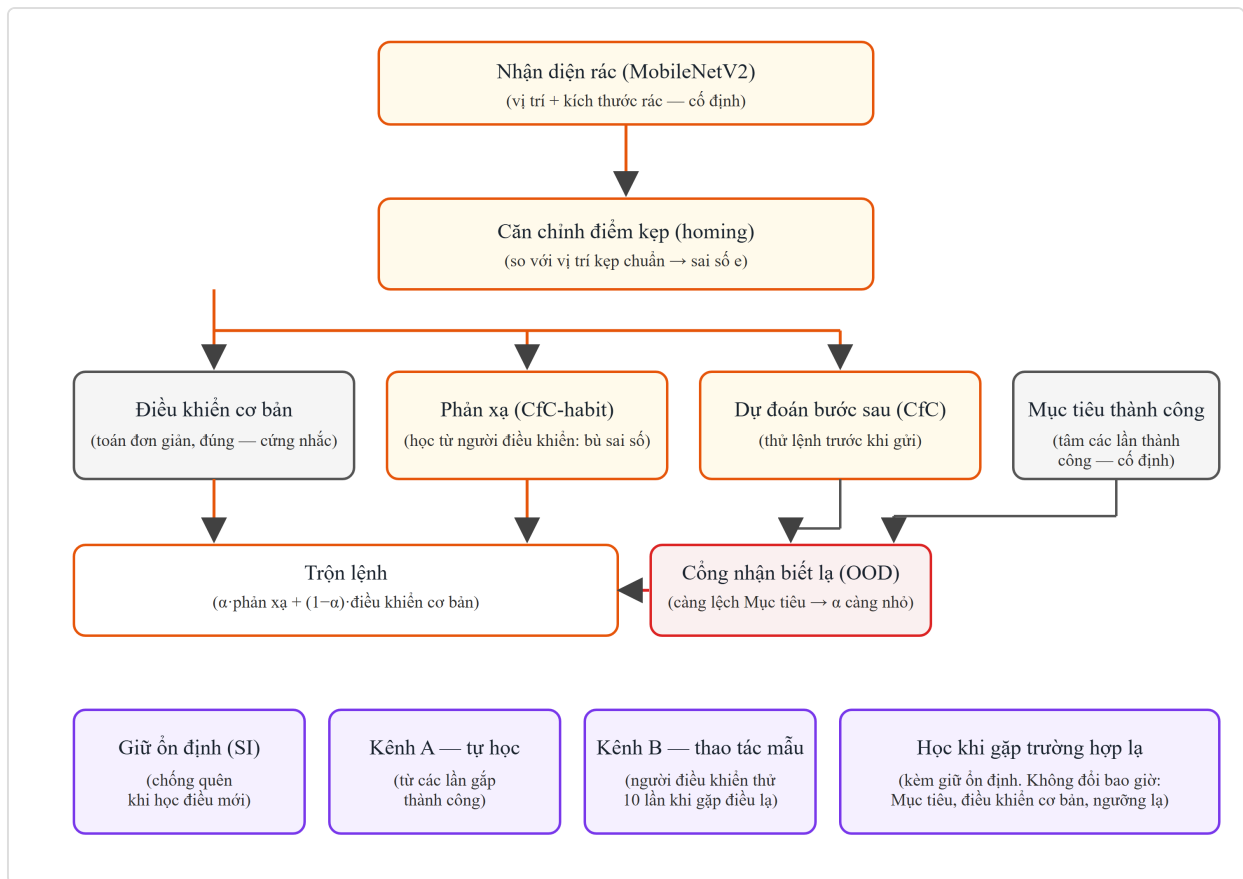
nhận ra điều bất thường, nhận ra bất thường mới kịp điều chỉnh, và nhờ vậy học được liên tục. Hai việc cụ thể: (a) thử lệnh trước khi gửi, chọn lệnh có kết quả tốt nhất; (b) khoảng cách giữa dự đoán đó với Mục tiêu thành công càng lớn, tình huống càng lạ — và nó báo song song cùng Cổng nhận biết lạ.

Kèm theo là tầng an toàn: **Cổng nhận biết lạ (OOD gate)**: trạng thái hiện tại so với Mục tiêu thành công (tâm các lần thành công — cố định, không học), sinh α — trọng số tin tưởng phản xạ. Khi gặp dữ liệu ngoài phân phối, α về 0: robot quay về điều khiển giải tích thuần — không bao giờ để mạng nơ-ron tự lái một mình.

Chuỗi vận hành: robot tìm rác (SEARCH — camera nhận diện) → đi tới gần (NAV — Nav2 định vị, tránh vật cản) → căn chỉnh điểm nắm (ALIGN — hai mạng cùng tham gia, pha trộn với điều khiển cơ bản theo α) → hạ tay, khớp kẹp, xác nhận bằng 4 mức vật lý thuần: lực và góc khớp từ servo, giám sát khi nâng; tầng này không dùng mạng nơ-ron — nếu mạng sai, tầng xác nhận vẫn bắt được, mở kẹp và thử lại tối đa 3 lần (GRASP) → nâng, chống trượt (LIFT) → bỏ rác vào thùng (DROP) → tìm rác tiếp.

Học lại cũng đi trên hai kênh: tự động (các lần thành công được sàng lọc bằng lực servo) và thao tác mẫu (10 lần người điều khiển thử trên miền dữ liệu lạ); cả hai đều cập nhật cục bộ kèm SI $[\lambda \Sigma \omega(W - W_0)^2]$ để không quên nhiệm vụ cũ. Mục tiêu thành công, ngưỡng lạ, điều khiển giải tích và lỗi nhận diện cố định trong suốt quá trình học — vì chúng là "cột mốc", không phải bài học.

Bản thiết kế này là **hướng nghiên cứu tiếp nối**: hành trình kéo dài từ V0 (robot thật) → V2.1 (đạt chuẩn benchmark) → V2.5.2 (ứng dụng thực tế), với mục tiêu hoàn thành trong **6 tuần** khi được hỗ trợ tiếp tục.



Hình H: Sơ đồ "bộ não" của robot — điều khiển và học. Nhận diện rác (MobileNetV2) → căn chỉnh điểm kẹp (homing) → bộ não gồm điều khiển cơ bản, phản xạ CfC-habit, dự đoán bước sau (CfC); Mục tiêu thành công cố định + Cổng nhận biết lạ (OOD) quyết định trọng số α để trộn lệnh; lớp học dưới cùng: giữ ổn định (SI), Kênh A tự học, Kênh B thao tác mẫu, học khi gặp trường hợp lạ.

8. Cách sử dụng, vận hành

8.1. [V0 — Robot thật] Vận hành phần cứng theo cử chỉ (demo)

Chạy demo điều khiển bằng cử chỉ bàn tay người:

```
python demo_grasp.py
```

8.1.2. [V0 — Robot thật] Vận hành mô hình thế giới + CEM trên robot thật

```
python robot_planner.py --goal <file> --model cfc --checkpoint <ckpt>
```

Chuỗi hoạt động: camera → bộ mã hóa → CEM mô phỏng 300 chuỗi × 30 vòng → chọn hành động tối ưu → servo thực hiện → kiểm tra vị trí — phát hiện nắm thành công.

```

Step 0: cost=1.124523
CEM iter 1/5: cost=0.342302 (best=0.342302)
CEM iter 3/5: cost=0.247082 (best=0.247082)
CEM iter 5/5: cost=0.211920 (best=0.211920)
Plan (H=3): {1: 320, 2: 47, 4: 603, 5: 355, 6: 909, 7: 947, 8: 408, 9: 688}
Position: S2:cmd=289 actual=288 err=1 GRASP | S4:cmd=310 actual=318 err=8 GRASP | S7:cmd=501 actual=507 err=6 GRASP
✅ All 3 servos at target — grasp detected
  
```

Hình I: Log terminal thật — CEM lập kế hoạch chạy trên robot V0: bước 0 → vòng CEM → kế hoạch $H=3$ → báo GRASP

8.2. [V2.1 — Benchmark mô phỏng] Tái lập kết quả đánh giá

```
python eval.py --config-name=pusht policy=<ckpt> --seed=3072 --config-name=pusht
```

Kết quả tái lập 3 hạt giống trên 2 GPU (T4/RTX 5090): 92%, 98%, 94% → $94.7\% \pm 3.1\%$ (Push-T).
Mã nguồn và trọng số huấn luyện công khai:

Thành phần	Liên kết
 Mã nguồn	github.com/thoan4965-ui/hybrid-mamba-2-attention-world-model
 Trọng số đã huấn luyện	huggingface.co/hhian/checkpoints

*Phân định rõ hai kết quả: Mục 8.1 là kết quả [V0 — robot thật] — tay bionic tự chế, dữ liệu tự thu nhỏ (8900 frame, 1 camera): đây là kiểm chứng nguyên lý (độ trôi $34\times$ thấp hơn AR, nắm chạy được chuỗi CEM thật), **không phải kết quả thống kê tái lập được** vì dữ liệu nhỏ và chỉ trên 1 mô hình robot. Mục 8.2 là kết quả [V2.1 — benchmark mô phỏng] với dữ liệu chuẩn (20K episode), 3 hạt giống $\times$ 2 GPU — tái lập được đầy đủ: $94.7\% \pm 3.1\%$ (Push-T).*



Hình J: Ảnh thật robot đang nắm chai nước trong hộp thu dữ liệu

9. Tài liệu tham khảo

#	Nguồn
[1]	LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence.
[2]	Maes, L., Le Lidec, Q., Scieur, D., LeCun, Y., Balestrieri, R. (2026). LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. arXiv 2603.19312.
[3]	Hasani, R. et al. (2022). Closed-form Continuous-time Neural Networks. Nature Machine Intelligence.
[4]	Dao, T. & Gu, A. (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. ICML 2024.
[5]	Balestrieri, R. & LeCun, Y. (2025). LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. arXiv 2511.08544.
[6]	Ma, C. & Najarian, K. (2025). Rethinking the long-range dependency in Mamba/SSM and transformer models. arXiv 2509.04226.
[7]	Huang, Y. (2026). VJEPA: Variational Joint Embedding Predictive Architectures as Probabilistic World Models. arXiv 2601.14354.
[8]	Gu, A. & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv 2312.00752.
[9]	Lieber, O. et al. (2024). Jamba: A Hybrid Transformer-Mamba Language Model.
[10]	Li, Y. et al. (2026). TransMamba: A Sequence-Level Hybrid Transformer-Mamba Language Model. AAAI 2026.
[11]	Liu, X. et al. (2020). Neural SDE: Stabilizing Neural ODE Networks with Stochasticity. NeurIPS 2020.
[12]	Rob Knight (2022). DexHand V1.0: Open-Source Dexterous Humanoid Robot Hand. GitHub.
[13]	Xu, H. et al. (2025). MambaLite-Micro: Memory-Optimized Mamba Inference on MCUs. arXiv 2509.05488.
[14]	Chiang, H.-Y. et al. (2025). Quamba: A Post-Training Quantization Recipe for Selective State Space Models. ICLR 2025.